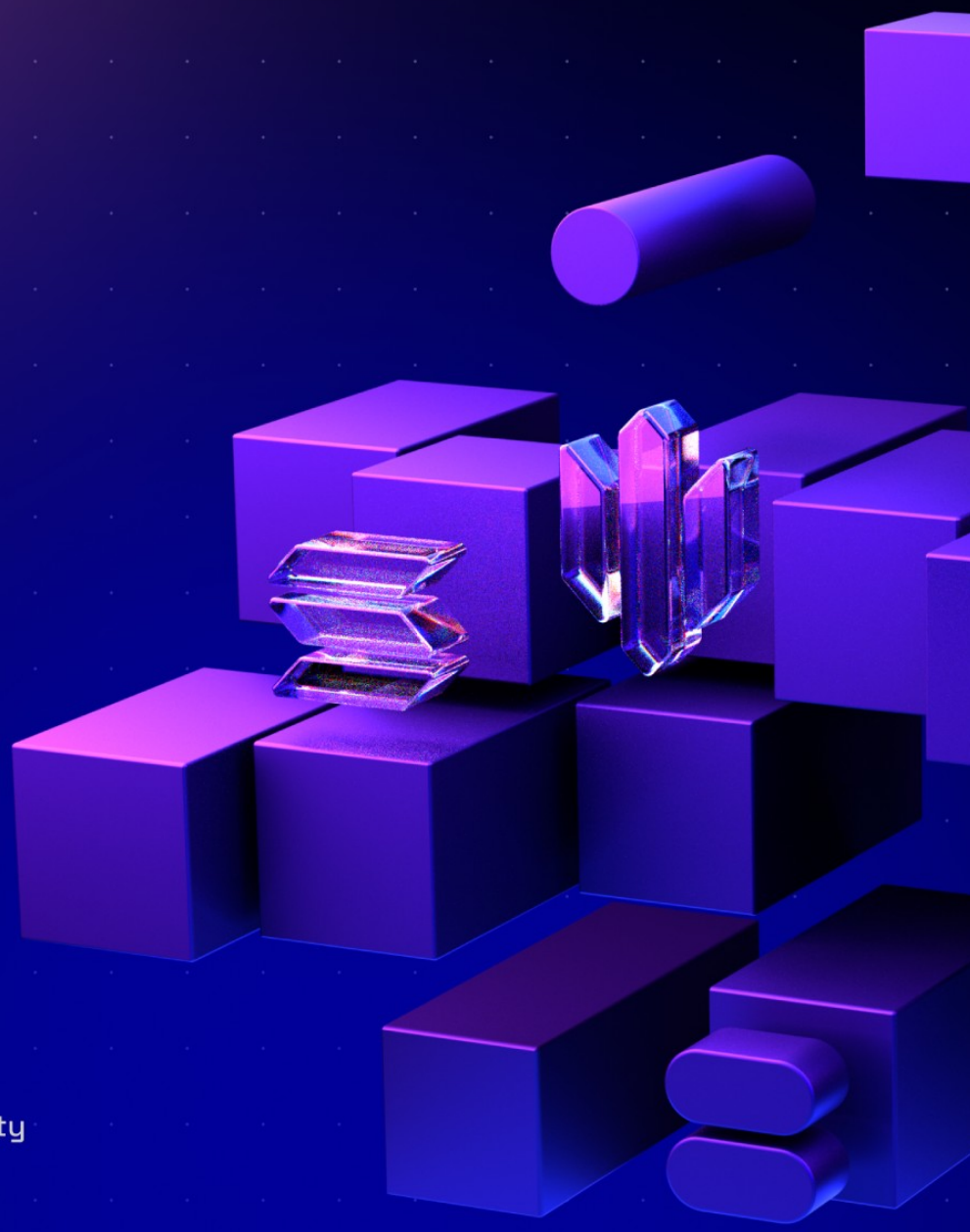
ackee
blockchain security

Windfall

Gamifying DeFi

20.12.2024



Ackee Blockchain Security

Contents

1. Document Revisions . . . 3
2. Overview . . . 4
2.1. Ackee Blockchain Security . . . 4
2.2. Audit Methodology . . . 5
2.3. Finding Classification . . . 6
2.4. Review Team . . . 8
2.5. Disclaimer . . . 8
3. Executive Summary . . . 9
Revision 1.0 . . . 9
Revision 1.1 . . . 11
4. Findings Summary . . . 12
Report Revision 1.0 . . . 14
Revision Team . . . 14
System Overview . . . 14
Trust Model . . . 14
Findings . . . 15
Appendix A: How to cite . . . 39

1. Document Revisions

1.0	Final Report	16.12.2024
1.1	Fix Review	20.12.2024

2. Overview

This document presents our findings in reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Audit Methodology

The Ackee Blockchain Security auditing process follows a routine series of steps:

1. **Code review**
    a. High-level review of the specifications, sources, and instructions provided to us to make sure we understand the project's size, scope, and functionality.
    b. Detailed manual code review, which is the process of reading the source code line-by-line to identify potential vulnerabilities. We focus mainly on common classes of Solana program vulnerabilities, such as:

       missing ownership checks, missing signer authorization, signed CPI of unverified programs, cosplay of Solana accounts, missing rent exemption assertion, bump seed canonicalization, incorrect accounts closing, casting truncation, numerical precision errors, arithmetic overflows or underflows.

    c. Comparison of the code and given specifications, ensuring that the program logic correctly implements everything intended.
    d. Review of best practices to improve efficiency, clarity, and maintainability.
2. **Testing and automated analysis**
    a. Run client's tests to ensure that the system works as expected, potentially write missing unit or fuzzy tests using our testing framework Trident.
3. **Local deployment + hacking**
    a. The programs are deployed locally, and we try to attack the system and break it. There is no specific strategy here, and each project's attack attempts are unique to its implementation.

2.3. Finding Classification

A *Severity* rating of each finding is determined as a synthesis of two sub-ratings: *Impact* and *Likelihood*. It ranges from *Informational* to *Critical*.

If we have found a scenario in which an issue is exploitable, it will be assigned an impact rating of *High*, *Medium*, or *Low*, based on the direness of the consequences it has on the system. If we haven't found a way, or the issue is only exploitable given a change in *configuration* (system settings or parameters, such as deployment scripts, compiler configurations, using multi-signature wallets for owners, etc.) or given a change in the codebase, then it will be assigned an impact rating of *Warning* or *Info*.

Low to *High* impact issues also have a *Likelihood*, which measures the probability of exploitability during runtime.

The full definitions are as follows:

Severity

		Likelihood			
		High	**Medium**	**Low**	**N/A**
Impact	**High**	Critical	High	Medium	-
	Medium	High	Medium	Low	-
	Low	Medium	Low	Low	-
	Warning	-	-	-	Warning
	Info	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue cannot be exploited given the current code and/or `configuration`, but could be a security vulnerability if these were to change slightly. If we haven't found a way to exploit the issue given the time constraints, it might be marked as a "Warning" or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations. Another example is that the issue would be security-related if code or `configuration` was to change.

Likelihood

- **High** - The issue is exploitable by virtually anyone under virtually any circumstance.
- **Medium** - Exploiting the issue currently requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

The following table lists all contributors to this report. For authors of the specific revision, see the “Revision team” section in the respective “Report revision” chapter.

Member’s Name	Position
Andrej Lukačovič	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We’ve put our best effort to find all vulnerabilities in the system, however our findings shouldn’t be considered as a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

3. Executive Summary

The Windfall is an application designed to gamify DeFi. The project consists of two on-chain Solana programs. Vault, designed to collect assets deposited by users and Raffle responsible for distributing rewards generated from the yield farming of the assets collected from users.

Revision 1.0

Windfall engaged Ackee Blockchain Security to perform a security review of the Windfall protocol with a total time donation of 7 engineering days in a period between December 6 and December 13, 2024, with Andrej Lukačovič as the lead auditor.

The audit was performed on the commit `ffbdda`[1] and `5c5455`[2] respectively. The scope was as follows:

- Raffle Program, excluding external dependencies;
- Vault Program, excluding external dependencies.

We began our review by exploring the materials provided by the client. In this case, the client had prepared numerous resources for the project, including an introduction video, pitch deck, GitBook documentation, demo, and GitHub README&s. While the GitHub READMEs contain important information about the source code, it would be beneficial to enhance the official documentation with a more detailed description of the functionality.

After this initial introduction to the project, we proceeded with the manual review. The project consists of two main components: the Raffle, responsible for distributing prizes to users, and the Vault, which collects assets deposited by users.

During the manual review of the Vault program, we paid special attention to:

- ensuring users' assets are always kept in the most secure manner;
- ensuring no unauthorized withdrawals of users' assets are possible;
- ensuring undesired operations by authorities cannot occur;
- ensuring the protocol adheres to permissionless practices as much as possible;
- ensuring correct data validation is in place.

During the manual review of the Raffle program, we paid special attention to:

- ensuring that there is no unauthorized handling of assets;
- ensuring that the raffle is fair to all users;
- ensuring the program is correctly and securely configured;
- ensuring the program adheres to permissionless practices as much as possible.

Our review resulted in 15 findings, ranging from Info to High severity. The most severe one is H1.

Ackee Blockchain Security recommends Windfall:

- ensure the development process always adheres to best security practices;
- ensure that Solana programs are thoroughly and robustly tested;
- ensure the assets deployed by users are always protected;
- ensure the raffle program operates fairly at all times;
- ensure the security of all secret keys that act as authorities for accounts or programs;
- address all reported issues promptly and effectively.

See Report Revision 1.0 for the system overview and trust model.

Revision 1.1

The fix review was completed on commits `9f69a9`[3] and `abbe94`[4]. The implementation of fixes occurred across multiple incremental commits, with this commit hash representing the final state.

The majority of reported issues have been resolved. The remaining acknowledged issues include responses from the client.

Ackee Blockchain Security recommends Windfall:

- ensure public raffles are initialized correctly;
- minimize and clearly communicate centralization risks to users.

[1] full commit hash: `ffbdda78c0b71c08580c965c0a14f8f5b6309e08`

[2] full commit hash: `5c54559ef2b3ffe029cc1acd88688e824f54cc31`

[3] full commit hash: `9f69a94277507ff304ed01eef09b5c55c56a297a`

[4] full commit hash: `abbe94b695bc42ef31874bd4551773f0dc49fcea`

4. Findings Summary

The following section summarizes findings we identified during our review. Unless overridden for purposes of readability, each finding contains:

- *Description*
- *Exploit scenario* (if severity is low or higher)
- *Recommendation*
- *Fix* (if applicable).

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	1	2	1	4	7	15

Table 2. Findings Count by Severity

Findings in detail:

Finding title	Severity	Reported	Status
H1: Repetitive vault creation and closing can drain the payer account	High	1.0	Fixed
M1: Excessive centralization in asset management	Medium	1.0	Acknowledged
M2: Insufficient restrictions for public raffles	Medium	1.0	Acknowledged
L1: Problematic access to the user's funds after the vault type is not supported anymore	Low	1.0	Fixed

Finding title	Severity	Reported	Status
W1: Prizes are not aligned with pool assets	Warning	1.0	Fixed
W2: Automatic evaluation of account's size	Warning	1.0	Fixed
W3: Prizes can be added while the raffle is in process	Warning	1.0	Fixed
W4: Misconfigured Merkle root causes cross-raffle redeems	Warning	1.0	Fixed
I1: Vault type authority transfer not possible to cancel	Info	1.0	Fixed
I2: Raffle can start in the past	Info	1.0	Fixed
I3: `UserRecord` cannot be closed	Info	1.0	Fixed
I4: Unused code / Code quality	Info	1.0	Fixed
I5: Reserve and pool accounts cannot be closed	Info	1.0	Fixed
I6: Prizes with zero payout are possible	Info	1.0	Acknowledged
I7: Raffle security depends on the Feed random number generator	Info	1.0	Acknowledged

Table 3. Table of Findings

Report Revision 1.0

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

The Windfall Project is an application designed to gamify DeFi. Users can deposit their assets into the Vault program, additionally these assets are utilized in other DeFi projects on Solana to farm yield. By participating in games, users can collect points, which can then be redeemed in the Raffle program to draw and distribute rewards generated from utilizing the deposited assets.

Trust Model

Both programs are significantly centralized, and these concerns are further detailed in the specific issues (e.g. M1).

For the Vault program, an authority controls the VaultType, allowing them to withdraw assets from the vault and reallocate them to other yielding protocols. The authority also has the ability to modify several critical configuration flags at their will.

In the Raffle program, the authority can similarly toggle configuration flags. Additionally, this authority is responsible for ensuring sufficient funds in the raffle vaults for users to claim prizes.

Findings

The following section presents the list of findings discovered in this revision.

H1: Repetitive vault creation and closing can drain the payer account

High severity issue

Impact:	High	Likelihood:	Medium
Target:	new_vault.rs, close_vault.rs	Type:	Data validation

Description

The Vault program allows an additional signer during the `NewVault` instruction invocation to cover the rent for the new vault on behalf of the user. However, the `CloseVault` instruction can be invoked in such a way that the rent is returned to the user.

The following sequence demonstrates the issue:

- payer covers the rent for the new vault;
- vault is closed by the user;
- rent is returned to the user.

If the payer is not the same as the user in the `NewVault` instruction, this effectively results in the rent amount being transferred from the payer to the user.

Exploit scenario

Alice is a malicious user, and Bob is the application backend wallet. The exploit proceeds as follows:

- Alice requests vault creation from the application;
- Bob's wallet (as payer) covers the rent fees for Alice's vault account;
- Alice immediately closes the vault using the `CloseVault` instruction;

- the rent is returned to Alice instead of Bob.

Since the `NewVault` instruction is constructed in the application backend and Bob's wallet automatically signs the transaction, Alice can exploit this repeatedly, draining Bob's wallet.

Recommendation

Restrict rent payment to the `user_authority` only, as they will be able to retrieve the rent upon vault closure.

Fix 1.1

The issue has been fixed by restricting rent payment to `user_authority` only.

Go back to Findings Summary

M1: Excessive centralization in asset management

Medium severity issue

Impact:	Medium	Likelihood:	Medium
Target:	-	Type:	Trust model

Description

The Solana programs exhibit significant centralization in their asset management and control mechanisms. While the complexity of implementing cross-protocol asset distribution logic is acknowledged, the current implementation grants excessive administrative control.

Exploit scenario

Vault program

Alice is a regular user who deposits funds into the Vault program. Bob is the administrator.

- Bob withdraws all assets from the vault;
- Alice attempts to withdraw her funds but cannot because the vault is empty;
- Bob has no on-chain obligation to redeposit the funds or distribute rewards;
- Bob locks the `VaultType` by arbitrarily setting the `is_locked` flag, preventing Alice from initiating the deactivation process for withdrawals;
- Alice's funds remain inaccessible due to Bob's centralized control.

Raffle program

Alice is a raffle participant. Bob is the administrator.

- Bob controls the Merkle root validation, determining who can participate;
- Bob sets the `is_public` flag arbitrarily, controlling raffle eligibility;
- Bob creates a raffle with prizes that do not match the vault amounts;
- Bob sets the `is_active` flag arbitrarily, deactivating the raffle before completion;
- Alice cannot redeem her tickets or participate in the prize drawing.

Recommendation

Communicate the centralization risks to the users. Implement strong security measures to ensure the safety of users' assets. Improve the decentralization of both programs.

Acknowledgment 1.1

The issue was acknowledged with the following client response:

> *The team is aware of this matter, and we plan to first automate its operation off-chain as a windSOL LST, and then implement it on-chain.*
>
> — Windfall Team

Go back to Findings Summary

M2: Insufficient restrictions for public raffles

Medium severity issue

Impact:	High	Likelihood:	Low
Target:	new_raffle.rs, redeem.rs	Type:	Data validation

Description

The Raffle program allows for two types of raffles, public and private. Private raffles are gated by eligibility, while the public raffles are meant for marketing purposes as the client has stated in their documentation:

> *Public raffle: a raffle with no participant restrictions. Mainly used for marketing purposes.*
>
> — Windfall Team

The implementation lacks sufficient restrictions for the public raffle. The following code demonstrates that public raffles do not verify user eligibility against a Merkle root. While this aligns with the intended use case, the absence of verification enables unrestricted participation and ticket redemption through multiple wallets. This design allows a single user to potentially redeem all available tickets.

Listing 1. Excerpt from Raffle program

```
    if !raffle.is_public {
        // Verification for private raffle
    } else {
        // Verify the number of tickets allowed per user
        require!(
            raffle.max_tickets_per_user >= tickets_allocated,
            ErrorCode::InvalidNumTickets
        );
    }
```

Exploit scenario

Alice could exploit a raffle marked as public with deposited funds by:

- calling the redeem instruction up to `max_tickets_per_user` to allocate tickets;
- calling the draw instruction to participate in the raffle;
- winning prizes despite potential ineligibility.

Recommendation

Remove this feature or, allow this feature only on devnet.

Acknowledgment 1.1

The issue was acknowledged with the following client response:

> *We are aware that a public raffle can be instantly exploited if someone develops a bot, and we have no intention of offering tokens with economic value as prizes. Additionally, while there are no concrete plans to operate a public raffle, we have decided not to remove it for now. When providing this program to external parties, we will thoroughly address the concerns raised in M2 in the documentation.*
>
> — Windfall Team

Go back to Findings Summary

L1: Problematic access to the user's funds after the vault type is not supported anymore

Low severity issue

Impact:	Low	Likelihood:	Medium
Target:	deactivate.rs, roll_over_vault_type.rs, withdraw.rs	Type:	Logic error

Description

It is expected that a `VaultType` may become unsupported and eventually closed. However, an unsupported `VaultType` creates challenges for users attempting to withdraw their funds.

When `instant_deactivation` is disabled, users with assets in an unsupported `VaultType` must complete the following steps to withdraw:

1. initiate the deactivation process;
2. wait for the deactivation process completion, allowing their vault to be marked as Inactive;
3. withdraw their assets.

This process presents several issues:

- the deactivation process requires the current timestamp to be lower than `deactivation_lock_start`, which cannot be guaranteed if the vault has not been rolled over to the next season;
- users must wait for the deactivation process to complete before the vault can be marked as Inactive, despite the `VaultType` being unsupported;
- the process must be repeated for each user who did not initiate

deactivation in the previous season, requiring continuous rolling of unsupported `VaultType` until all users have withdrawn.

Exploit scenario

Consider the following values (selected artificially for simplicity):

Listing 2. Excerpt from Vault program

```
vt.season_start = 10_000;
vt.season_duration = 1_000;
vt.deactivation_lock_window = 300;

let now = 50_000
```

and code snippet from deactivation:

Listing 3. Pseudo-code from Vault program (deactivate.rs)

```
let season_end = vt.season_start + vt.season_duration;
let deactivation_lock_start = season_end - vt.deactivation_lock_window;
let now = Clock::get()?.unix_timestamp;

/// ! In order to proceed with deactivation,
/// ! now must be smaller than deactivation_lock_start.
require!(now < deactivation_lock_start, ErrorCode::DeactivationLocked);

/// season_end = 10_000 + 1_000 = 11_000
/// deactivation_lock_start = 11_000 - 300 = 10_700
```

In this scenario, Alice holds assets in an unsupported `VaultType`. To withdraw her assets, she must initiate the deactivation process. However, the system requires that the current timestamp is less than `deactivation_lock_start` for the deactivation to proceed. If the timestamp exceeds this value, Alice will be unable to complete her withdrawal, causing a delay until the deactivation process is finished. Alice may not be aware that the vault must be rolled over to the next season for the deactivation to be successful.

Recommendation

Implement one of the following solutions:

- instruction to enable instant deactivation;
- introduce a new `VaultType` parameter indicating support status, allowing instant withdrawals for deprecated vault types.

Fix 1.1

The issue has been fixed by adding the `unsupport_vault_type` instruction, which will force `instant_deactivation` to be enabled.

Go back to Findings Summary

W1: Prizes are not aligned with pool assets

Impact:	Warning	Likelihood:	N/A
Target:	add_prize_to_raffle.rs	Type:	Trust model

Description

The `add_prize_to_raffle` instruction allows the authority to define prizes without requiring corresponding asset deposits in the raffle pool. This misalignment between declared prizes and available assets means that winners may be unable to claim their designated prizes due to insufficient pool balance.

Recommendation

Modify the `add_prize_to_raffle` instruction to require simultaneous deposit of prize assets into the pool when prizes are added.

Fix 1.1

The issue has been fixed by transferring assets to the pool when a new prize is added.

Go back to Findings Summary

W2: Automatic evaluation of account's size

Impact:	Warning	Likelihood:	N/A
Target:	states.rs	Type:	Code quality

Description

For multiple accounts the account size is automatically evaluated using:

Listing 4. Excerpt from Raffle

```
pub const SIZE: usize = std::mem::size_of::<Self>();
```

This appoach can create problems with deserialization in the following scenario:

- the current version of the program is deployed to mainnet, expecting a specific account size and structure;
- development continues, and additional fields are added to the account;
- a new version of the program is deployed to Mainnet;
- with the new version of the program (which includes new fields in the account), it will no longer be possible to deserialize accounts created with the older version of the program, as these accounts do not contain the new fields that the updated program expects.

Recommendation

Implement additional reserved space in the account structure to accommodate future field additions.

Fix 1.1

The issue has been fixed, by adding a reserved space to the account size.

Go back to Findings Summary

W3: Prizes can be added while the raffle is in process

Impact:	Warning	Likelihood:	N/A
Target:	add_prize_to_raffle.rs	Type:	Data validation

Description

As described in M1, the administrator has the ability to modify the activity status of a raffle at will. This capability allows them to deactivate the raffle and add prizes during the redeem and draw phases, which may compromise the fairness of the process for users.

Recommendation

Implement logic that restricts adding prizes only during the period before the raffle begins.

Fix 1.1

The issue has been fixed by allowing prizes to be added only before the raffle begins.

Go back to Findings Summary

W4: Misconfigured Merkle root causes cross-raffle redeems

Impact:	Warning	Likelihood:	N/A
Target:	redeem.rs	Type:	Data validation

Description

The raffle's Merkle tree implementation lacks proper raffle-specific binding in its node generation. When multiple raffles (e.g., Raffle A and Raffle B) are created, an incorrectly configured Merkle root allows users to redeem tickets across different raffles. Specifically, if Raffle A uses Raffle B's Merkle root, users eligible for Raffle B could redeem tickets in Raffle A.

Exploit scenario

The current implementation creates Merkle nodes using only the user's public key and ticket allocation:

Listing 5. Excerpt from Raffle

```
let node = hashv(&[
    &[0u8],
    &hashv(&[&ctx.accounts.user_authority.key().to_bytes()]).to_bytes(),
    &tickets_allocated.to_le_bytes(),
]);

require!(
    verify(proof, ctx.accounts.raffle.merkle_root, node.to_bytes()),
    ErrorCode::InvalidProof
);
```

Alice is eligible for Raffle B but not Raffle A. Due to misconfigured Merkle roots, Alice can use her proof from Raffle B to redeem tickets in Raffle A, potentially claiming prizes she is not entitled to.

Recommendation

Include the public key of the raffle as part of the hashed values used for the node hash. This would ensure that each raffle's Merkle tree is uniquely tied to its specific raffle, preventing scenarios where tickets from one raffle could be redeemed in another due to a shared Merkle root.

Fix 1.1

The issue was fixed by including the public key of the raffle as part of the hashed values used for the node hash.

Go back to Findings Summary

I1: Vault type authority transfer not possible to cancel

Impact:	Info	Likelihood:	N/A
Target:	-	Type:	Code quality

Description

The Vault program implements a two-step logic for transferring `VaultType` authority, which is considered secure. However, the current authority cannot cancel the transfer process once initiated.

Recommendation

- allow the current authority to accept the transfer process, resulting in no change to the authority field;
- implement the same logic in the Raffle program as well.

Fix 1.1

The issue has been fixed by allowing the current vault type authority to cancel the authority transfer process. For the Raffle program, the authority transfer process is not implemented, and this is the client's response:

> *Authority transfer process can be canceled by the original authority via cancel_vault_type_authority_transfer instruction. Raffle accounts have a shorter life cycle and we decided not to implement the authority transfer feature to the raffle program.*
>
> — Windfall Team

Go back to Findings Summary

I2: Raffle can start in the past

Impact:	Info	Likelihood:	N/A
Target:	new_raffle.rs	Type:	Data validation

Description

A raffle can start in the past. There does not seem to be any reason to allow this.

Recommendation

Ensure that the start time is at least the current timestamp.

Fix 1.1

The issue has been fixed by ensuring raffles can only start in the future.

Go back to Findings Summary

I3: `UserRecord` cannot be closed

Impact:	Info	Likelihood:	N/A
Target:	-	Type:	Code quality

Description

The Raffle program does not allow users to close their `UserRecord` accounts, even after the raffle is no longer relevant.

Recommendation

Add logic to support closing the `UserRecord` after the raffle has ended. It is important to allow the `UserRecord` account to be closed only after the raffle is no longer relevant, as early closing could result in double-spend issues.

Fix 1.1

The issue has been fixed by adding a new instruction `close_user_record`, which allows users to close their user record accounts. The instruction is only available after the raffle has ended.

Go back to Findings Summary

I4: Unused code / Code quality

Impact:	Info	Likelihood:	N/A
Target:	-	Type:	Code quality

Description

Some parts of the code are redundant. For example:

- `cooldown_window` is not used (this is mentioned in the comments in the source code);
- `CloseVault` does not need to include `vault_type` as an account input;
- `new_authority` in the `NominateVaultTypeAuthority` instruction does not need to be an account but can be a `Pubkey` on the instruction input.

Recommendation

Remove unused parts of the source code.

Fix 1.1

The issue has been fixed by removing the unused code.

Go back to Findings Summary

I5: Reserve and pool accounts cannot be closed

Impact:	Info	Likelihood:	N/A
Target:	close_vault_type.rs	Type:	Code quality

Description

Once the `VaultType` is closed, the `Reserve` and `Pool` Token Accounts will become unclosable.

Recommendation

Close the pool and reserve accounts while closing the `VaultType`.

Fix 1.1

This issue has been fixed in commit `59f6f8`[1], which was added after the audit. The fix includes closing both the pool and reserve token accounts when closing the vault type.

Go back to Findings Summary

I6: Prizes with zero payout are possible

Impact:	Info	Likelihood:	N/A
Target:	add_prize_to_raffle.rs	Type:	Data validation

Description

The raffle allows for prizes with a payout of zero. While this may be considered a feature, users should be made aware of it. Based on communication with the client, it is understood from various discussions that they are aware of this issue and consider it a feature.

Recommendation

If this behavior is not desired, add data validation to ensure that the payout cannot be zero.

Acknowledgment 1.1

The issue was acknowledged with the following client response:

> *This is by design, so while we leave it as it is, we will definitely explain it in the documents when we offer the program to external parties.*
>
> — Windfall Team

Go back to Findings Summary

I7: Raffle security depends on the Feed random number generator

Impact:	Info	Likelihood:	N/A
Target:	-	Type:	N/A

Description

During the `Draw` instruction, random numbers are obtained from the Feed Protocol RNG. The project also has a public audit report available.

Once the random number is obtained from the Feed program, it is used in the Wyhash algorithm to generate additional numbers from the input seed (where the seed is the random number). However, Wyhash does not introduce any additional randomness and is fully predictable. This means that the unpredictability of the raffle is entirely dependent on the cryptographic security of the Feed program. Consequently, the random number generator (RNG) must satisfy cryptographic security requirements, such as:

- Next-bit test: If a sequence of bits generated by the RNG is known, it should be computationally infeasible to predict the next bit with a probability significantly greater than 50%; Source
- State compromise resistance: If the internal state of the RNG is leaked, it must be impossible to reconstruct previously generated random numbers or streams; Source

The first property ensures that it is impossible to predict how the raffle will behave in the future based on current or past data. The second property ensures that even if the RNG state is compromised, it remains impossible to obtain future raffle results by reconstructing random numbers.

Recommendation

Ensure that the currently implemented random number generator is secure.

Acknowledgment 1.1

The issue was acknowledged with the following client response:

> *We understand that we depend on Feed Protocol RNG security and quality for now. We plan to continue exploring and considering custom implementations or transitions if better RNG solutions become available.*
>
> — Windfall Team

Go back to Findings Summary

[1] full commit hash: `59f6f8cf08c1ec2921aa47cb397090df9eefb4a3`

Appendix A: How to cite

Please cite this document as:

> Ackee Blockchain Security, Windfall: Gamifying DeFi, 20.12.2024.

ackee
blockchain security

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz